Using Claude Code With OpenCode Go

A practical backend tutorial for `claude-o-go`

`claude-o-go`

June 18, 2026

Contents

1 Goal

This tutorial explains the setup in this repository and why it works. The goal is to keep the Claude Code CLI experience while sending model requests to OpenCode Go instead of Anthropic's Claude API.

Claude Code remains the harness: terminal UI, tool calling, file edits, permissions, prompt handling, and the agent loop. OpenCode Go is the backend that receives the actual model request.

2 What You Have

This directory contains:

- `.env`: your OpenCode Go API key.
- `claude-o-go`: the launcher script you run instead of `claude`.
- `opencode-go-claude-proxy.mjs`: the local proxy between Claude Code and OpenCode Go.
- `opencode-go-claude-example.mjs`: a direct API example that calls OpenCode Go without Claude Code.

The normal command is:

```
cd /home/user/Desktop/opencode-go-w-claude
./claude-o-go
```

For a one-shot prompt:

```
./claude-o-go -p "Reply with exactly: OK"
```

3 Run `claude-o-go` From Anywhere

You do not need to change into this repository every time, and you do not need to copy it into each new project. Install it once and use `claude-o-go` from any project directory.

3.1 One-Time Setup

Make the launcher executable if needed:

```
chmod +x /home/user/Desktop/opencode-go-w-claude/claude-o-go
```

Create a symlink in a directory already on your `PATH`:

```
mkdir -p ~/.local/bin
ln -s /home/user/Desktop/opencode-go-w-claude/claude-o-go \
  ~/.local/bin/claude-o-go
```

Make sure `~/.local/bin` is on your `PATH`:

```
export PATH="$HOME/.local/bin:$PATH"
```

Then reload your shell:

```
source ~/.bashrc   # or: source ~/.zshrc
```

3.2 Why This Works

The launcher resolves its own symlink with `readlink -f`. It then changes into the real install directory, where `.env` and `opencode-go-claude-proxy.mjs` live. That makes the launcher work no matter where you call it from.

3.3 Optional Shell Profile Configuration

If you would rather not keep the key in `.env`, export it once in your shell profile:

```
export OPENCODE_GO_API_KEY="sk-..."
export OPENCODE_GO_MODEL="qwen3.7-plus"
```

The launcher picks up `OPENCODE_GO_API_KEY` from the environment and skips reading `.env`. The model is no longer hardcoded in the launcher; define `OPENCODE_GO_MODEL` in your shell profile so it applies everywhere.

3.4 Daily Usage

From any project directory:

```
cd /path/to/any/project
claude-o-go
```

For a one-shot:

```
claude-o-go -p "Explain this repository in one paragraph."
```

Switch models for a single invocation:

```
OPENCODE_GO_MODEL=minimax-m3 claude-o-go
```

4 Why a Proxy Is Needed

OpenCode Go exposes several models through an Anthropic-compatible Messages API. It is a routing and billing layer, not a model host. Each request is routed to the model vendor's own API.

Examples:

```
qwen3.7-plus
minimax-m3
qwen3.7-max
```

Claude Code validates model names locally before sending requests. If you try to call Claude Code directly with an OpenCode Go model, Claude Code rejects it before the request is useful:

```
ANTHROPIC_BASE_URL=https://opencode.ai/zen/go/v1 \
ANTHROPIC_API_KEY=... \
claude --model qwen3.7-plus
```

The proxy solves this by letting Claude Code think it is using a normal Claude alias such as `sonnet`, while the proxy rewrites the outgoing request to use the OpenCode Go model.

5 Model Compatibility

Each upstream provider implements its own Anthropic-compatible endpoint, so compatibility varies. Claude Code sends a full Anthropic-style payload, including tools, cache control, tool choice, and Anthropic beta headers.

Model id	**Provider**	**Works with Claude Code harness?**
`qwen3.7-plus`	Alibaba (Qwen)	Yes
`minimax-m3`	MiniMax	Yes
`glm-5.2`	Z.ai (Zhipu AI)	No; invalid API parameter
`glm-5.1`	Z.ai (Zhipu AI)	No; same as `glm-5.2`
`kimi-k2.7-code`	Moonshot AI	No; rejects tool function name format
`deepseek-v4-pro`	DeepSeek	No; expects OpenAI tool shape

Models from the same vendor generally behave like the tested one. Unverified models should be tested before relying on them.

Practical recommendation:

```
export OPENCODE_GO_MODEL="qwen3.7-plus"   # or minimax-m3
```

6 Request Flow

When you run:

```
./claude-o-go -p "Reply with exactly: OK"
```

the flow is:

1. `claude-o-go` reads the OpenCode Go API key from `.env`.
2. It starts the local proxy at `http://127.0.0.1:4141`.

3. It sets `ANTHROPIC_BASE_URL=http://127.0.0.1:4141`.
4. It starts Claude Code with `claude -bare -model sonnet`.
5. Claude Code sends a request to `/v1/messages`.
6. The proxy changes the request body model to the OpenCode Go model.
7. The proxy forwards the request to `https://opencode.ai/zen/go/v1/messages`.
8. OpenCode Go returns the model response.
9. The proxy streams the response back to Claude Code.

7 Important Files

7.1 `claude-o-go`

The launcher exports the API key, model, and local base URL, then starts Claude Code:

```
export OPENCODE_GO_API_KEY="$api_key"
export OPENCODE_GO_MODEL="$model"
export ANTHROPIC_BASE_URL="http://127.0.0.1:${proxy_port}"
claude --bare --model "$claude_model_alias" "$@"
```

Important defaults and environment variables:

- `OPENCODE_GO_MODEL`: required; no hardcoded launcher default.
- `OPENCODE_GO_PROXY_PORT=4141`.
- `CLAUDE_CODE_MODEL_ALIAS=sonnet`.
- `CLAUDE_CODE_OPENCODE_GO_BARE=1`.

7.2 `opencode-go-claude-proxy.mjs`

The proxy's core rewrite is:

```
body.model = model
```

For generation requests, the proxy streams OpenCode Go's response back to Claude Code. It handles Node stream backpressure and aborts the upstream request if Claude Code closes the local connection, so a transient `connection interrupted` message should not take down the proxy process.

The proxy also implements endpoints Claude Code probes:

```
HEAD /
HEAD /v1
GET /v1/models
POST /v1/messages/count_tokens
POST /v1/messages
```

8 Local Checks

Before pushing changes, run:

```
npm run check
```

That syntax-checks both JavaScript entry points.

9 Verification

Run:

```
./claude-o-go -p "Reply with exactly: OK"
```

Look for proxy log lines like:

```
OpenCode Go Claude proxy listening on http://127.0.0.1:4141/v1 -> qwen3.7-plus
POST /v1/messages
POST /messages -> 200 as qwen3.7-plus
OK
```

The key line is:

```
POST /messages -> 200 as qwen3.7-plus
```

That means Claude Code sent a request to the local proxy, the proxy forwarded it to OpenCode Go, OpenCode Go returned HTTP 200, and the upstream model was `qwen3.7-plus`.

10 Common Problems

10.1 `There's an issue with the selected model`

Use the launcher:

```
./claude-o-go
```

Do not call `claude -model qwen3.7-plus` directly.

10.2 Double `/v1/v1/messages`

`ANTHROPIC_BASE_URL` should point to the proxy root:

```
http://127.0.0.1:4141
```

It should not include `/v1`, because Claude Code appends `/v1/messages` itself.

10.3 Claude Code Shows `sonnet`

That is expected. Claude Code sees `sonnet`; OpenCode Go receives `qwen3.7-plus`. The proxy log is the source of truth.

11 Mental Model

```
Claude Code CLI
  thinks model = sonnet
  sends Anthropic request
        |
        v
Local proxy
  rewrites model = qwen3.7-plus
  forwards request
        |
        v
OpenCode Go
  runs the actual model
  returns response
        |
        v
Claude Code CLI
  displays the answer
```

The Claude Code user experience stays the same, but the model backend is OpenCode Go.